HƯỚNG DẪN THAO TÁC SỬ DỤNG AUTHENTICATION API LÀM ĐỂ TEST

Chúng ta sẽ dùng API /register-test mà bạn đã thiết kế để tự động hóa toàn bộ quá trình cấp quyền.

Bước 1: Tạo toàn bộ các tài khoản bằng API Register-Test

Trong Postman, mở Collection, tạo một thư mục tên là **0. Setup Data**. Sử dụng API POST {{baseUrl}}/api/v1/auth/register-test để chạy 5 Request sau (API này không cần Token):

Request tạo Admin:

{"email": "test_admin@university.edu.vn", "password": "Password@123",
"fullName": "QA Admin", "systemRole": "ADMIN"}

Request tạo Manager:

{ "email": "manager@ou.edu.vn", "password": "password123", "fullName": "Trưởng Phòng NCKH", "systemRole": "MANAGER" }

Request tạo Trưởng khoa:

{ "email": "dept_it@ou.edu.vn", "password": "password123", "fullName": "Trưởng Khoa IT", "systemRole": "DEPT_HEAD" }

Request tạo Researcher (Chủ nhiệm đề tài):

{ "email": "researcher1@ou.edu.vn", "password": "password123", "fullName": "Giảng viên A", "systemRole": "RESEARCHER" }

Request tạo Chuyên gia Hội đồng:

{ "email": "expert1@ou.edu.vn", "password": "password123", "fullName": "Chuyên gia B", "systemRole": "COUNCIL" }

Bước 2: Cơ chế Auto-Token Injection: Bạn tạo API **POST {{baseUrl}}/api/v1/auth/login**

Trong mục **Tests** của Postman, bắt buộc phải có đoạn mã JavaScript này:

```
if (pm.response.code === 200) {
   var jsonData = pm.response.json();
   pm.environment.set("jwt_token", jsonData.token);
   console.log("Token đã được cập nhật cho Role: " + jsonData.user.systemRole);
}
```

Bước 3: Quy trình Test xoay vòng

Bây giờ, hãy cấu hình toàn bộ thư mục chứa 40 API còn lại (Topics, Councils...) ở mục **Authorization** chọn **Bearer Token** và điền {{jwt_token}}.

1. Mở Postman.
2. Mở API **Login**, nhập email của researcher1@ou.edu.vn, bấm Send.
3. Ví dụ mở API **POST /topics** (Tạo đề tài), bấm Send. (Thành công vì token hiện tại là của Researcher).
4. Mở lại API **Login**, đổi email thành dept_it@ou.edu.vn, bấm Send. (Token tự động bị ghi đè thành quyền Trưởng khoa).
5. Ví dụ mở API **PATCH /topics/{id}/status** để duyệt đề tài vừa tạo, bấm Send. (Thành công).

BÁO CÁO KỸ THUẬT: CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU KIỂM THỬ GIAO DIỆN LẬP TRÌNH ỨNG DỤNG
(API TEST DATA ENGINEERING & SPECIFICATION)
PHÂN HỆ: QUẢN LÝ ĐỀ TÀI (TOPICS) VÀ HỘI ĐỒNG XÉT DUYỆT (COUNCILS)
Người thực hiện: Chuyên gia Kiểm thử Cấp cao & Thiết kế API
Ngày lập báo cáo: 31/03/2026

---

1. TỔNG QUAN VÀ PHƯƠNG PHÁP LUẬN KIỂM THỬ KHOA HỌC

Báo cáo này cung cấp hướng dẫn chuyên sâu về kỹ thuật thiết kế bộ dữ liệu kiểm thử (Test Data Generation) cho các điểm cuối (Endpoints) thuộc nhóm Topics và Councils đã được xác định trên tài liệu OpenAPI/Swagger UI.

Để đảm bảo độ bao phủ mã (Code Coverage) và độ bao phủ nghiệp vụ (Business Coverage) đạt mức tối đa, quá trình kiến tạo dữ liệu kiểm thử bắt buộc áp dụng các phương pháp luận kiểm thử hộp đen (Black-box Testing) tiêu chuẩn quốc tế (được định nghĩa bởi ISTQB và IEEE):

1. **Phân lớp tương đương (Equivalence Partitioning - EP):** Phân chia miền dữ liệu đầu vào thành các lớp hợp lệ (Valid) và không hợp lệ (Invalid) để giảm thiểu số lượng ca kiểm thử nhưng vẫn đảm bảo độ tin cậy.
2. **Phân tích giá trị biên (Boundary Value Analysis - BVA):** Tập trung dữ liệu vào các ngưỡng giới hạn của kiểu dữ liệu (ví dụ: độ dài chuỗi tối đa 255 ký tự, kinh phí tối thiểu/tối đa).
3. **Kiểm thử chuyển đổi trạng thái (State Transition Testing - STT):** Đặc biệt quan trọng đối với thực thể Đề tài (Topic) vốn được quản trị bởi Máy trạng thái (State Machine). Dữ liệu phải giả lập các bước chuyển trạng thái hợp lệ và các nỗ lực chuyển trạng thái phi logic.
4. **Kiểm thử dựa trên rủi ro bảo mật (RBAC & Security Testing):** Thiết lập các mã thông báo (JWT) đại diện cho nhiều vai trò khác nhau để kiểm chứng nguyên lý Phân quyền Vi mô (Micro-Authorization).

---

2. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU KIỂM THỬ - NHÓM API ĐỀ TÀI (TOPICS) - THƯƠNG

Nhóm API này quản lý vòng đời của tài sản cốt lõi trong hệ thống. Dữ liệu kiểm thử phải phản ánh sự khắt khe trong việc bảo vệ tính toàn vẹn của hồ sơ khoa học.

2.1. Nhóm API Khởi tạo và Cập nhật (POST /api/v1/topics/, PUT /api/v1/topics/{id})

- **Nguyên lý áp dụng:** EP, BVA.
- **Thiết kế Dữ liệu Hợp lệ (Happy Path):**
    - Tải trọng (Payload) chứa đầy đủ các trường bắt buộc (title_vn, research_type, research_field, duration_months, expected_budget).
    - Sử dụng mã thông báo (Token) của tác nhân RESEARCHER.

- **Thiết kế Dữ liệu Không hợp lệ (Negative Testing):**
    - *Vi phạm kiểu dữ liệu:* duration_months mang giá trị âm hoặc giá trị thập phân; expected_budget mang kiểu chuỗi (String) thay vì số thực (Decimal).
    - *Vi phạm giá trị biên:* title_vn mang chuỗi rỗng (Empty String) hoặc chuỗi vượt quá 255 ký tự để kiểm chứng cơ chế Hibernate Validator (JSR-380) có bắt giữ ngoại lệ và trả về mã HTTP 400 Bad Request hay không.
- **Kiểm thử ràng buộc trạng thái (Dành riêng cho PUT):**
    - Dữ liệu đầu vào: Truyền {id} của một đề tài đang ở trạng thái PENDING_DEPT hoặc APPROVED.
    - Kỳ vọng: Hệ thống phải từ chối cập nhật và ném ra mã HTTP 409 Conflict.

2.2. API Chuyển đổi Trạng thái (PATCH /api/v1/topics/{id}/status)

- **Nguyên lý áp dụng:** State Transition Testing (STT), RBAC.
- **Thiết kế Dữ liệu Hợp lệ:**
    - Luồng 1: Đề tài trạng thái DRAFT, tác nhân RESEARCHER gửi payload yêu cầu chuyển sang PENDING_DEPT.
    - Luồng 2: Đề tài trạng thái PENDING_DEPT, tác nhân DEPT_HEAD gửi payload yêu cầu chuyển sang DEPT_APPROVED kèm theo feedback_message.
- **Thiết kế Dữ liệu Vi phạm Cấu trúc Trạng thái:**
    - *Cố tình vượt cấp (State Bypassing):* Đề tài trạng thái DRAFT, tác nhân RESEARCHER cố tình gửi yêu cầu chuyển thẳng sang APPROVED. Kỳ vọng hệ thống từ chối (Mã 409).
    - *Vi phạm đặc quyền:* Đề tài trạng thái PENDING_DEPT, tác nhân RESEARCHER cố tình gửi lệnh chuyển trạng thái thay vì DEPT_HEAD. Kỳ vọng Tầng Bảo mật đánh chặn (Mã 403 Forbidden).

2.3. Nhóm API Tệp Đính kèm (POST, GET /api/v1/topics/{id}/attachments...)

- **Nguyên lý áp dụng:** Boundary Value Analysis (Kích thước tệp), Security (Định dạng tệp).
- **Thiết kế Dữ liệu Kiểm thử:**
    - *Payload Hợp lệ:* Tệp định dạng chuẩn (.pdf, .docx) với kích thước nằm trong ngưỡng cho phép (ví dụ: 2MB).
    - *Payload Vi phạm Bảo mật:* Cố tình đính kèm tệp thực thi (.exe, .sh) hoặc tệp chứa mã kịch bản (.html, .js) để kiểm tra bộ lọc kiểm tra nội dung (Content-Type Validation) của máy chủ.
    - *Payload Vi phạm Dung lượng:* Tệp có kích thước vượt giới hạn cấu hình spring.servlet.multipart.max-file-size (ví dụ: 50MB) để kiểm tra ngoại lệ HTTP 413 Payload Too Large.

2.4. Nhóm API Truy xuất (GET /api/v1/topics, /api/v1/topics/{id}..., DELETE)

- **Thiết kế Dữ liệu Phân trang (Pagination):** Truyền các tham số truy vấn (Query Parameters) ?page=0&size=20, ?page=-1 (Kỳ vọng lỗi 400), ?size=1000 (Kiểm chứng giới hạn kích thước trang tối đa để chống tấn công cạn kiệt tài nguyên cơ sở dữ liệu).
- **Thiết kế Dữ liệu Xóa (DELETE):** Cung cấp {id} của đề tài đang ở trạng thái DRAFT (Kỳ vọng 204 No Content). Cung cấp {id} của đề tài đang ở trạng thái PENDING_DEPT (Kỳ vọng 409 Conflict - Không được phép xóa hồ sơ đã nộp).

---

3. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU KIỂM THỬ - NHÓM API HỘI ĐỒNG (COUNCILS) - THỊNH

Nhóm API này phức tạp về mặt logic nghiệp vụ kết nối đa thực thể. Quá trình kiểm thử tập trung vào việc xác minh tính toàn vẹn tham chiếu và các quy tắc xung đột lợi ích (Conflict of Interest).

3.1. API Khởi tạo Hội đồng (POST /api/v1/councils/)

- **Nguyên lý áp dụng:** BVA (Kiểm thử yếu tố thời gian).
- **Thiết kế Dữ liệu Kiểm thử:**
  - *Thời gian hợp lệ:* meeting_date là một ngày trong tương lai (ví dụ: 2026-05-01).
  - *Thời gian phi logic:* meeting_date là một ngày trong quá khứ. Cần kiểm tra xem Validation Service có sử dụng chú thích @FutureOrPresent để chặn dữ liệu này hay không.

3.2. API Phân công Thành viên (POST /api/v1/councils/{id}/members)

- **Nguyên lý áp dụng:** Business Rule Validation, Equivalence Partitioning.
- **Thiết kế Dữ liệu Hợp lệ (Cấu trúc nhân sự chuẩn):** Payload chứa danh sách 5 đối tượng với các vai trò không trùng lặp (PRESIDENT, SECRETARY, REVIEWER_1, REVIEWER_2, MEMBER).
- **Thiết kế Dữ liệu Vi phạm Quy tắc Nghiệp vụ (Cực kỳ quan trọng):**
  - *Xung đột lợi ích (Conflict of Interest):* Trong danh sách userId truyền lên, cố tình chèn userId của chính Chủ nhiệm Đề tài (Investigator) đang được hội đồng này xét duyệt. Kỳ vọng hệ thống ném ngoại lệ logic nghiệp vụ (HTTP 400 hoặc 409) từ chối thao tác.
  - *Vi phạm cấu trúc:* Truyền lên 2 chuyên gia cùng mang vai trò PRESIDENT. Kỳ vọng hệ thống nhận diện và từ chối.
  - *Dữ liệu trùng lặp:* Truyền cùng một userId hai lần trong một mảng. Kỳ vọng thuật toán xử lý dữ liệu tập hợp (Set) tại tầng Service ném ngoại lệ.

3.3. API Phân công Đề tài (POST /api/v1/councils/{id}/topics)

- **Nguyên lý áp dụng:** State Transition Testing.
- **Thiết kế Dữ liệu Kiểm thử:**
  - *Payload Hợp lệ:* Mảng topicId chứa các đề tài đang ở trạng thái DEPT_APPROVED.

- *Payload Vi phạm:* Mảng topicId chứa một đề tài đang ở trạng thái DRAFT hoặc REJECTED. Hệ thống phải kiểm tra trạng thái của từng đề tài trong mảng và thực hiện Rollback toàn bộ giao dịch nếu phát hiện 1 đề tài không hợp lệ.

3.4. Nhóm API Hủy phân công (DELETE /api/v1/councils/{id}/topics/{topicId}, /members/{userId})

- **Nguyên lý áp dụng:** Phụ thuộc tồn tại (Existence Dependency).
- **Thiết kế Dữ liệu Kiểm thử:**
  - Cố tình gọi API xóa một chuyên gia (DELETE Member) khi chuyên gia đó **đã nộp Phiếu chấm điểm (Evaluation)**. Dựa trên thiết kế cơ sở dữ liệu ON DELETE CASCADE, nếu xóa thành viên, phiếu điểm sẽ bị mất. Do đó, tầng Service phải chặn hành động này (Trả về HTTP 409 Conflict) để bảo vệ dữ liệu học thuật. Hành động xóa chỉ hợp lệ khi chuyên gia chưa có phát sinh dữ liệu chấm điểm.

4. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU KIỂM THỬ: NHÓM USER MANAGEMENT - THƯƠNG

Nhóm API này do Quản trị viên (ADMIN) hoặc Quản lý cấp cao (MANAGER) vận hành, đòi hỏi tính bảo mật và toàn vẹn tham chiếu dữ liệu tuyệt đối.

1. Nhóm API Khởi tạo Tài khoản (POST /api/v1/users/researchers, /managers, /dept-heads)

- **Nguyên lý áp dụng:** Equivalence Partitioning, Referential Integrity, RBAC.
- **Thiết kế Dữ liệu Hợp lệ (Happy Path):**
  - Tải trọng (Payload) cung cấp đầy đủ thông tin: email, fullName, academicTitle, và departmentId.
  - Biến departmentId phải tham chiếu đến một bản ghi thực sự tồn tại trong bảng departments.
  - Sử dụng mã thông báo (JWT) của ADMIN.
- **Thiết kế Dữ liệu Không hợp lệ (Negative Testing):**
  - *Vi phạm tính Duy nhất (Unique Constraint):* Cố tình cung cấp một email đã tồn tại trong cơ sở dữ liệu. Kỳ vọng hệ thống ném ngoại lệ và trả về mã HTTP 400 hoặc 409 (Tùy thuộc cấu hình GlobalExceptionHandler).
  - *Vi phạm Định dạng (Format Validation):* Cung cấp chuỗi email sai định dạng biểu thức chính quy (Regex) (ví dụ: admin_at_university.com). Kỳ vọng hệ thống từ chối với mã HTTP 400 Bad Request.
  - *Vi phạm Toàn vẹn Tham chiếu (Foreign Key Violation):* Truyền departmentId mang giá trị ảo (ví dụ: 999999). Kỳ vọng hệ thống bắt lỗi DataIntegrityViolationException.
- **Kiểm thử Phân quyền (Security Testing):**

    - Sử dụng JWT của RESEARCHER cố tình gọi API /api/v1/users/managers. Kỳ vọng hệ thống trả về HTTP 403 Forbidden.

2. API Cập nhật Trạng thái Tài khoản (PATCH /api/v1/users/{id}/status)

- **Nguyên lý áp dụng:** State Transition Testing.
- **Thiết kế Dữ liệu Kiểm thử:**
    - *Luồng Khóa tài khoản:* Truyền tải trọng { "isActive": false } cho một tài khoản đang hoạt động.
    - *Luồng Kích hoạt lại:* Truyền tải trọng { "isActive": true } cho tài khoản đã bị khóa.
    - *Luồng Tự sát (Self-lock Prevention):* Sử dụng JWT của ADMIN (Ví dụ ID: 1), cố tình gọi API này để truyền tham số {id} là 1 với trạng thái false. Kỳ vọng Tầng Dịch vụ có logic nghiệp vụ ngăn chặn Quản trị viên tự khóa chính mình để tránh làm liệt hệ thống (Trả về HTTP 409 Conflict hoặc 400 Bad Request).

3. API Truy xuất Danh sách (GET /api/v1/users/)

- **Nguyên lý áp dụng:** Boundary Value Analysis.
- **Thiết kế Dữ liệu Phân trang:**
    - Truyền tham số page=0&size=20&sort=userId,desc để kiểm tra khả năng phân trang và sắp xếp đúng chuẩn.
    - Cố tình truyền size=-5 hoặc chuỗi ký tự vào tham số số học. Kỳ vọng Spring Data xử lý lỗi an toàn bằng cách sử dụng cấu hình phân trang mặc định (Fallback to default) hoặc ném lỗi 400.

---

5. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU KIỂM THỬ: NHÓM MEETING MINUTES - Thịnh

Nhóm API này đại diện cho nút thắt cổ chai học thuật (Academic Bottleneck) của hệ thống. Biểu đồ đối tượng (Object Graph) tại đây phức tạp nhất vì nó phụ thuộc vào Đề tài, Hội đồng, và toàn bộ Phiếu đánh giá.

1. API Đệ trình Biên bản Hội đồng (POST /api/v1/minutes)

Đây là API đòi hỏi kỹ thuật giả lập trạng thái (State Mocking) phức tạp nhất.

- **Nguyên lý áp dụng:** Pre-condition Enforcement, ABAC (Attribute-Based Access Control).
- **Điều kiện tiên quyết bắt buộc (Môi trường Dữ liệu):**
    - Phải có một councilId tồn tại.
    - Hội đồng đó phải có các thành viên đã được phân công.
    - **Toàn bộ** các thành viên (trừ Thư ký) phải có bản ghi trong bảng evaluations với trạng thái SUBMITTED.
- **Thiết kế Dữ liệu Hợp lệ (Happy Path):**
    - Tải trọng chứa councilId, finalDecision (ví dụ: APPROVED_MINOR_REVISION), synthesizedComments(Tóm tắt ý kiến).

    - Sử dụng chính xác JWT của tài khoản đang mang vai trò SECRETARY của councilId tương ứng.
    - *Kỳ vọng:* Máy chủ tính toán đúng Điểm trung bình, lưu biên bản, chuyển trạng thái đề tài, trả về mã HTTP 201 Created.
- **Thiết kế Dữ liệu Vi phạm Tiền điều kiện (Negative Testing):**
    - *Chấm điểm chưa hoàn tất:* Giả lập cơ sở dữ liệu có 1 chuyên gia chưa nộp phiếu. Thư ký cố tình gọi API nộp biên bản. Kỳ vọng Tầng Dịch vụ thực thi nguyên lý Thất bại Nhanh (Fail-Fast), trả về HTTP 409 Conflict với thông báo "Chưa đủ điều kiện lập biên bản".
    - *Lập biên bản trùng lặp:* Cố tình gọi API này 2 lần liên tiếp cho cùng một councilId. Ràng buộc UNIQUEtrên cơ sở dữ liệu vật lý phải kích hoạt và trả về HTTP 409 Conflict.
- **Kiểm thử Ranh giới Ủy quyền (ABAC Testing):**
    - Sử dụng JWT của PRESIDENT (Chủ tịch) hoặc REVIEWER (Phản biện) thuộc cùng hội đồng đó để gọi API này. Mặc dù họ có chung nhóm quyền COUNCIL, biểu thức @PreAuthorize phải đánh chặn và trả về HTTP 403 Forbidden vì định danh vai trò nội bộ không phải Thư ký.

2. API Truy xuất Biên bản theo Đề tài (GET /api/v1/minutes/topic/{topicId})

- **Nguyên lý áp dụng:** Cross-reference Data Retrieval, Data Privacy.
- **Thiết kế Dữ liệu Kiểm thử:**
    - Truyền topicId của một đề tài đã hoàn tất hội đồng.
    - Truyền topicId của một đề tài đang ở trạng thái DRAFT hoặc PENDING_DEPT. Kỳ vọng API trả về HTTP 404 Not Found kèm thông báo "Biên bản chưa được lập".
- **Kiểm thử Quyền riêng tư (Privacy/Security Testing):**
    - Sử dụng JWT của RESEARCHER_A cố tình truy xuất topicId thuộc sở hữu của RESEARCHER_B.
    - *Kỳ vọng bảo mật:* Dù RESEARCHER_A có quyền gọi API này, nhưng Tầng Dịch vụ phải có logic đối khớp investigator_id với định danh trong Token. Nếu không khớp, trả về HTTP 403 Forbidden. Điều này ngăn chặn rò rỉ dữ liệu khoa học (Data Leakage) giữa các giảng viên.

6. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU: NHÓM DEPARTMENTS (KHOA/PHÒNG BAN) - TOÀN

Nhóm API này phục vụ việc truy xuất dữ liệu danh mục (Master Data) và phân luồng dữ liệu theo cấu trúc tổ chức (Organizational Scoping). Trọng tâm kiểm thử đặt tại cơ chế Kiểm soát Truy cập Dựa trên Thuộc tính (Attribute-Based Access Control - ABAC).

1. API Truy xuất Danh mục Toàn cục (GET /api/v1/departments/)

- **Nguyên lý áp dụng:** Access Control Validation, Cache Verification.
- **Thiết kế Dữ liệu Kiểm thử:**

- *Luồng Hợp lệ (Happy Path):* Sử dụng mã thông báo (JWT) của bất kỳ tác nhân nào đã xác thực. Kỳ vọng API trả về toàn bộ danh sách phòng ban có cờ is_active = true.
- *Luồng Kiểm chứng Trạng thái:* Can thiệp trực tiếp vào cơ sở dữ liệu vật lý (MySQL), cấu hình một Khoa thành is_active = false. Gọi lại API. Kỳ vọng mảng JSON phản hồi tuyệt đối không chứa Khoa vừa bị vô hiệu hóa, nhằm đảm bảo Frontend không hiển thị các đơn vị đã giải thể vào danh sách thả xuống (Dropdown List).

2. API Truy xuất Đề tài theo Khoa (GET /api/v1/departments/me/topics)

- **Nguyên lý áp dụng:** Data Isolation Testing (Kiểm thử Cách ly Dữ liệu Nhiều khách thuê).
- **Thiết kế Dữ liệu Hợp lệ (Phân vùng Dữ liệu):**
  - Chuẩn bị dữ liệu (Data Setup): Khởi tạo 05 đề tài thuộc Khoa Công nghệ Thông tin (CNTT) và 05 đề tài thuộc Khoa Kinh tế.
  - Thực thi: Sử dụng JWT của DEPT_HEAD thuộc Khoa CNTT.
  - Kỳ vọng: Tổng số phần tử trả về (totalElements) phải chính xác bằng 05. Tuyệt đối không được rò rỉ bất kỳ đề tài nào của Khoa Kinh tế.
- **Thiết kế Dữ liệu Không hợp lệ (Security Boundary):**
  - Sử dụng JWT của RESEARCHER hoặc COUNCIL. Kỳ vọng Tầng Bảo mật ném ngoại lệ AccessDeniedException và trả về mã HTTP 403 Forbidden, bảo vệ nghiêm ngặt ranh giới quản lý cấp cơ sở.

---

7. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU: NHÓM STATISTICS (THỐNG KÊ) - TOÀN

Nhóm API này (GET /api/v1/stats/summary) cung cấp dữ liệu tổng hợp (Aggregated Metrics) cho Bảng điều khiển (Dashboard). Điểm tử huyệt của nhóm API này là hiện tượng Bất đồng bộ Trạng thái (State Inconsistency) hoặc Lỗi tràn bộ đệm khi tập dữ liệu lớn.

- **Nguyên lý áp dụng:** Algorithmic Verification, Context-Aware Aggregation, Concurrency Simulation.
- **Thiết kế Dữ liệu Theo Ngữ cảnh (Context-Aware Testing):**
  1. Thuật toán của API này phải biến thiên theo Vai trò (Role) truyền vào từ JWT.
  2. *Trường hợp ADMIN/MANAGER:* Kỳ vọng kết quả đếm (Count) tổng hợp toàn bộ đề tài trên hệ thống phân theo từng trạng thái (DRAFT, PENDING_DEPT, APPROVED...).
  3. *Trường hợp DEPT_HEAD:* Kỳ vọng kết quả đếm chỉ tính trên tập hợp các đề tài có managing_department_id khớp với department_id của Phụ trách Khoa đó.
- **Thiết kế Dữ liệu Kiểm chứng Tính Thời gian thực (Real-time Synchronization):**
  1. Gọi API Thống kê lần 1, ghi nhận chỉ số pendingDeptCount (Ví dụ = 10).

2. Sử dụng API luồng nghiệp vụ (PATCH /api/v1/topics/{id}/status) để chuyển 01 đề tài từ DRAFT sang PENDING_DEPT.
3. Gọi lại API Thống kê lần 2.
4. *Kỳ vọng:* Chỉ số pendingDeptCount phải lập tức cập nhật thành 11. Thao tác này nhằm loại trừ lỗi do Bộ nhớ đệm (Cache) cấu hình sai tại Tầng Dịch vụ hoặc Tầng Trình diễn.

- **Kiểm thử Dữ liệu Trống (Zero-State Boundary):** Làm sạch cơ sở dữ liệu (Truncate bảng Topics). Gọi API Thống kê. Kỳ vọng hệ thống trả về cấu trúc JSON hợp lệ với toàn bộ các giá trị đếm bằng 0, thay vì ném ra ngoại lệ NullPointerException.

---

8. CHIẾN LƯỢC THIẾT KẾ DỮ LIỆU: NHÓM EVALUATIONS (ĐÁNH GIÁ/CHẤM ĐIỂM) - TOÀN

Nhóm API (POST /api/v1/evaluations) xử lý các phép toán định lượng khắt khe nhất của hệ thống. Mọi sai sót về làm tròn số (Rounding) hoặc kiểm tra biên (Boundary Check) đều gây ảnh hưởng trực tiếp đến kết quả khoa học.

- **Nguyên lý áp dụng:** Boundary Value Analysis (BVA), Equivalence Partitioning (EP), Idempotency Constraint Testing.

1. Phân tích Giá trị Biên đối với Điểm số (Score BVA)

Cấu trúc cơ sở dữ liệu sử dụng DECIMAL(5,2) cho các điểm thành phần. Hệ thống phải xác thực dữ liệu đầu vào (Payload Validation) một cách tuyệt đối trước khi ghi vào đĩa vật lý.

- **Dữ liệu Biên hợp lệ (Valid Boundaries):**
  - Truyền tải trọng với các điểm thành phần ở mức tối thiểu: 0.00.
  - Truyền tải trọng với các điểm thành phần ở mức tối đa theo từng tiêu chí (Ví dụ: score_urgency = 15.00, score_content = 25.00).
  - *Kỳ vọng:* Mã HTTP 201 Created. Tổng điểm (total_score) do máy chủ Backend tính toán phải khớp tuyệt đối với tổng các điểm thành phần.
- **Dữ liệu Biên không hợp lệ (Invalid Boundaries / Out of Bounds):**
  - *Luồng Âm:* Truyền score_urgency = -0.01.
  - *Luồng Vượt khung:* Nếu tiêu chí có điểm tối đa là 15, truyền score_urgency = 15.01 hoặc 16.00.
  - *Kỳ vọng:* Hibernate Validator (JSR-380) phải đánh chặn ngay tại Controller, trả về mã HTTP 400 Bad Request kèm thông báo lỗi trường dữ liệu, không được phép gọi xuống Tầng Dịch vụ.

2. Kiểm thử Ràng buộc Idempotency và Xung đột Lợi ích (Idempotency & Conflict)

- **Kiểm thử Tính Nộp một lần (Single-Submission Constraint):**
  - Tiền điều kiện: Chuyên gia (COUNCIL) đã nộp phiếu thành công cho Đề tài A.
  - Thực thi: Gửi lại chính xác Payload đó một lần nữa cho Đề tài A.

    - *Kỳ vọng:* Tầng Dịch vụ phải phát hiện chuyên gia này đã tồn tại bản ghi trong bảng evaluations. Trả về mã HTTP 409 Conflict với thông điệp "Mỗi thành viên chỉ được nộp một phiếu đánh giá cho mỗi đề tài".
- **Kiểm thử Ngữ cảnh Phân công (Unassigned Evaluation Attempt):**
    - Sử dụng JWT của COUNCIL_A. Cố tình gửi Payload chấm điểm cho topicId mà COUNCIL_A **không được phân công**.
    - *Kỳ vọng:* Trả về HTTP 403 Forbidden hoặc 400 Bad Request, bảo vệ nguyên tắc chỉ những chuyên gia trong quyết định thành lập hội đồng mới có quyền truy cập và chấm điểm hồ sơ.

PROMPT CHO TỰ ĐỘNG TẠO DATA

CÓ THỂ DỊCH LẠI QUA TIẾNG VIỆT. KHUYẾN NGHỊ NÊN DÙNG CHO CÁC IDEA AI CÓ THỂ TỰ ĐỘNG ĐỌC CODE. NẾU DÙNG AI CHATBOX BÌNH THƯỜNG THÌ PHẢI COPY TOÀN BỘ FILE CODE CHỨA API ĐƯỢC YÊU CẦU TRONG CONTROLLER BỎ VÀO.

“

 # SYSTEM PERSONA

Act as a Principal Quality Assurance Architect, Lead API Automation Engineer, and Data Scientist. You possess deep academic and empirical expertise in the Software Testing Life Cycle (STLC), Domain-Driven Design (DDD), and RESTful API Architecture.

 # CORE DIRECTIVE

Your primary objective is to conduct rigorous academic research, analyze the provided API specifications, and architect a comprehensive, edge-case-resilient Test Data Suite. You must not generate arbitrary data. Every test data entity must be mathematically, structurally, and logically justified based on established software engineering standards.

 # METHODOLOGICAL MANDATES

Before synthesizing any test data, you MUST analyze the input through the lens of globally recognized standards:

1. ISTQB Principles: Strictly apply Equivalence Partitioning (EP), Boundary Value Analysis (BVA), and State Transition Testing (STT).
2. ISO/IEC 25010 & IEEE 29119: Ensure the test data evaluates Functional Suitability, Performance Efficiency, and Security.
3. OWASP API Security Top 10: Inject controlled malicious payloads to verify Payload Validation and Broken Object Level Authorization (BOLA) defenses.

 # INPUT SPECIFICATIONS

Please analyze and generate test data for the following API:

- Endpoint URI: [INSERT_URI_HERE]
- HTTP Method:[INSERT_METHOD_HERE]
- Authorized Roles (RBAC/ABAC): [INSERT_ROLES_HERE]
- Request Payload Schema (JSON): [INSERT_SCHEMA_HERE]
- Business Rules & Constraints: [INSERT_BUSINESS_RULES_HERE]

 # EXPECTED OUTPUT STRUCTURE

Your response must strictly follow this taxonomy:

PHASE 1: ACADEMIC & METHODOLOGICAL JUSTIFICATION

Provide a concise theoretical analysis of the API's domain logic. Explain which testing techniques (EP, BVA, STT) are most applicable to this specific endpoint and why, referencing relevant IEEE or ISTQB standards.

PHASE 2: VALID TEST DATA SYNTHESIS (POSITIVE TESTING)

Construct precise JSON payloads representing the Happy Path. Ensure all mandatory fields meet the specified constraints. Provide the exact JSON object and the expected HTTP Status Code.

PHASE 3: BOUNDARY & INVALID TEST DATA SYNTHESIS (NEGATIVE TESTING)

Construct precise JSON payloads designed to violate structural and logical constraints. Focus on:

- Type Mismatches (e.g., String instead of Decimal).
- Boundary Violations (Exceeding max length, negative integers).
- Null/Empty injection for mandatory attributes.

Detail the expected HTTP Status Code (e.g., 400 Bad Request) and the expected behavior of the JSR-380 Validator.

PHASE 4: SECURITY & STATE VIOLATION DATA

Construct payloads or scenarios designed to test systemic defenses:

- State Machine Violations (e.g., submitting data when the entity is in an immutable state).
- Privilege Escalation (e.g., using a valid payload but executing under an unauthorized Role token).

PHASE 5: POSTMAN AUTOMATION IMPLEMENTATION

Provide a deterministic JavaScript snippet suitable for the Postman 'Pre-request Script' or 'Tests' tab. This script must programmatically validate the mathematical or structural integrity of the expected response.

 # STRICT FORMATTING & TONE CONSTRAINTS

- Tone: Highly academic, formal, objective, and professional.
- Terminology: Use exact computer science and software testing nomenclature.
- Absolute Prohibitions: DO NOT use any emojis or icons. DO NOT use colloquialisms. DO NOT use unnecessary quotation marks for emphasis. Focus entirely on high-density informational value and logical deduction.

“

<u>LƯU Ý: NHỚ THAY CÁC THÔNG SỐ CỦA API CẦN THIẾT KẾ DATA VÀO INPUT SPECIFICATIONS TRONG PROMPT</u>

KHUYẾN NGHỊ TRIỂN KHAI TRÊN MÔI TRƯỜNG POSTMAN

Để thực thi khối lượng dữ liệu kiểm thử đa dạng nêu trên một cách khoa học và tự động hóa, quy trình trên Postman cần tuân thủ các nguyên lý sau:

Thiết lập Kịch bản Tiền xử lý (Pre-request Scripts): Thay vì nhập cứng (Hardcode) các giá trị như tên đề tài hay ngày tháng, hãy sử dụng thư viện Faker.js tích hợp sẵn trong Postman để sinh dữ liệu ngẫu nhiên, đảm bảo các ràng buộc UNIQUE trong cơ sở dữ liệu không bị vi phạm khi chạy lại Collection nhiều lần.

// Ví dụ Pre-request script tạo mã đề tài ngẫu nhiên

1. pm.variables.set("randomTopicCode", "DT-" + pm.variables.replaceIn('{{$randomInt}}'));
2. **Sử dụng Trình chạy Tập hợp (Collection Runner) với Tệp Dữ liệu (Data-Driven Testing):** Đối với các trường hợp kiểm thử Vi phạm biên (Boundary) ở mục 2.1, hãy thiết lập một tệp .csv hoặc .json chứa hàng chục bộ dữ liệu lỗi khác nhau, cấu hình Postman Runner đọc tệp này để tự động bắn phá API nhằm xác minh sức chịu đựng của hệ thống.
3. **Kiểm định Chuỗi (Chaining Tests):** Luồng luân chuyển trạng thái (Workflow) đòi hỏi tính liên tục. Đảm bảo sử dụng thẻ Tests trong Postman để trích xuất topicId từ phản hồi của lệnh POST (Tạo đề tài) và lưu vào biến Môi trường, sau đó tự động truyền topicId này vào các lệnh PATCH (Chuyển trạng thái) kế tiếp.

HƯỚNG DẪN KỸ THUẬT TRIỂN KHAI TRÊN POSTMAN

Để biến các chiến lược trên thành tài nguyên tự động hóa (Automated Test Assets), Kỹ sư Kiểm thử (QA) cần thực thi theo trình tự sau trên Postman:

Bước 1: Thiết lập Trình tự Chuỗi (Test Chaining)

Không thể kiểm thử API POST /api/v1/minutes một cách độc lập. Cần tạo một thư mục (Folder) trong Postman Collection tên là [Flow] Council Minute Approval. Kéo thả các API theo đúng trật tự:

1. POST /auth/login (Lấy Token Quản lý).
2. POST /topics (Tạo đề tài).
3. PATCH /topics/.../status (Đẩy đề tài đến trạng thái chờ hội đồng).
4. POST /councils (Lập hội đồng).
5. POST /evaluations (Chạy vòng lặp 4 lần cho 4 chuyên gia để nộp phiếu).
6. POST /minutes (Lập biên bản).

Bước 2: Quản lý Trạng thái Ảo (Mocking State via Variables)

Sử dụng tab Tests của Postman sau mỗi lệnh gọi API thành công để trích xuất khóa chính và lưu vào bộ nhớ môi trường.

// Ví dụ sau khi tạo tài khoản Researcher thành công:

```
var response = pm.response.json();
pm.environment.set("test_researcher_id", response.userId);
```

Bước 3: Tự động hóa Kiểm thử Dữ liệu Biên (Data-Driven API Testing)
Tạo một tệp invalid_users.json chứa mảng các đối tượng (email sai định dạng, thiếu tên, tên chứa ký tự đặc biệt).
Sử dụng công cụ **Postman Collection Runner**, chọn thư mục chứa API POST /api/v1/users/researchers, nạp tệp JSON này vào. Viết mã trong tab Tests để khẳng định pm.response.to.have.status(400) cho mọi vòng lặp. Điều này giúp kiểm chứng sự bền bỉ của Tầng Validation (JSR-380) chỉ bằng một lần cấu hình.
Việc áp dụng bản thiết kế dữ liệu kiểm thử này bảo đảm hệ thống Backend được rà soát ở mức độ khắt khe nhất, loại trừ hoàn toàn các lỗ hổng về trạng thái nghiệp vụ và phân quyền trước khi tiến hành tích hợp mã nguồn Frontend.

HƯỚNG DẪN KỸ THUẬT TRIỂN KHAI TRÊN POSTMAN
Để nâng cao tính tự động hóa và độ tin cậy trong quá trình kiểm thử các nhóm API phức tạp này, Kỹ sư Đảm bảo Chất lượng (QA) cần thực hiện các thiết lập sau trên công cụ Postman:
Kiểm chứng Thuật toán bằng JavaScript (Tab Tests):
Đối với API POST /api/v1/evaluations, tuyệt đối không kiểm tra tổng điểm bằng mắt thường. Viết mã kiểm thử trong thẻ Tests của Postman để so khớp toán học:

```
pm.test("Toán học: Tổng điểm máy chủ trả về phải khớp với tổng điểm thành phần", function
() {
   var req = JSON.parse(pm.request.body.raw);
   var res = pm.response.json();

   var expectedTotal = req.scoreUrgency + req.scoreContent + req.scoreObjectives +
                req.scoreMethodology + req.scoreFeasibility +
                req.scoreCapacity + req.scoreProducts;

   // Xử lý sai số thập phân trong Javascript
   expectedTotal = Math.round(expectedTotal * 100) / 100;

   pm.expect(res.totalScore).to.eql(expectedTotal);
```

1. });
2. **Khẳng định Cấu trúc Dữ liệu (JSON Schema Validation):**
   Đặc biệt đối với API Statistics, cấu trúc phản hồi phải được kiểm chứng chặt chẽ (đảm bảo không trường nào trả về kiểu Null). Sử dụng thư viện tv4 hoặc Ajv tích hợp trong Postman để đối chiếu phản hồi JSON với một Khế ước Lược đồ (Schema Contract) định sẵn.